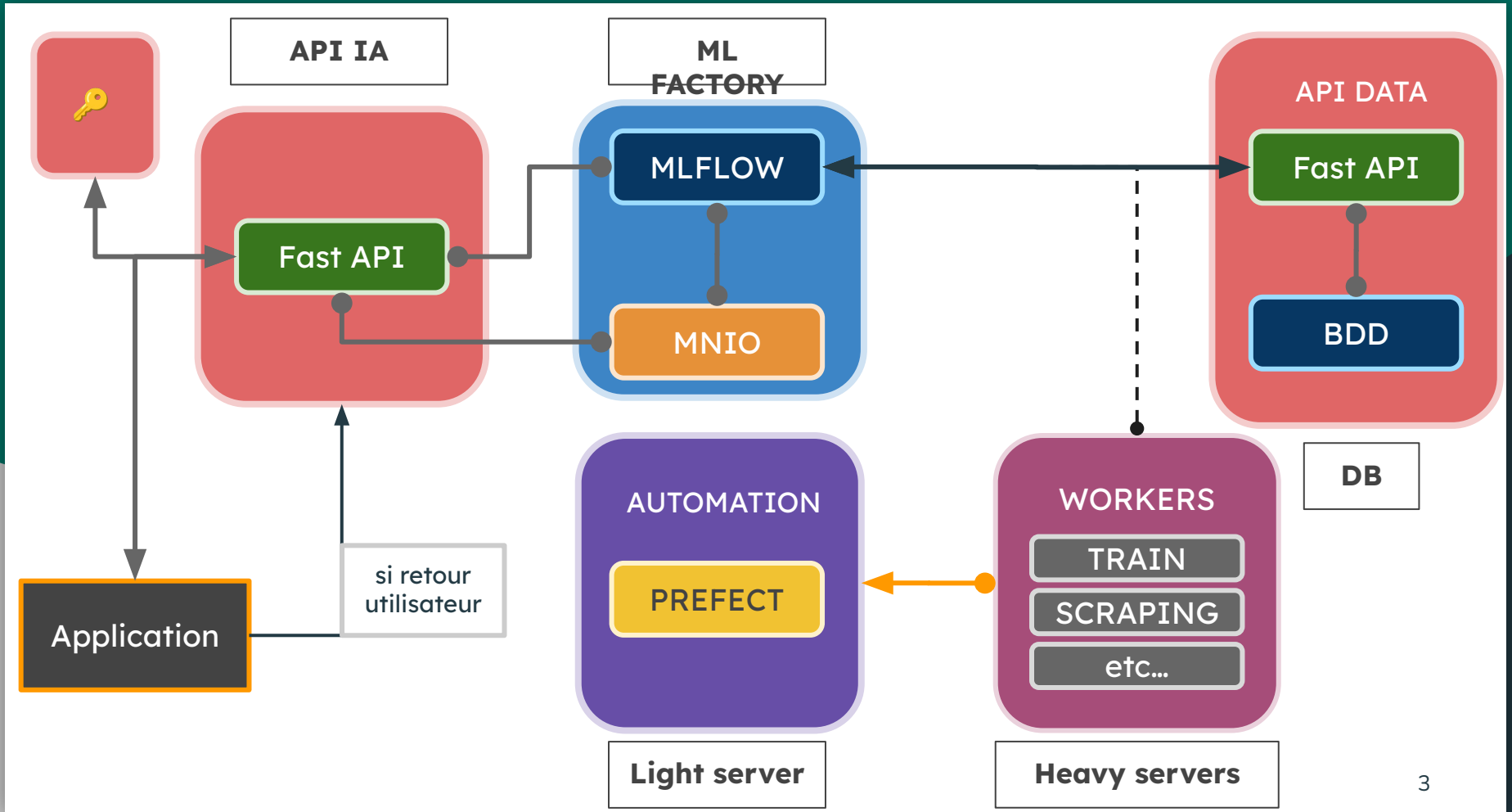
ML OPS Automation

A.SCHUTZ



Prefect documentation


API IA
ML FACTORY
API DATA
Fast API
MLFLOW
Fast API
MNIO
BDD
DB
AUTOMATION
PREFECT
WORKERS
TRAIN
SCRAPING
etc...
Application
si retour utilisateur
Light server
Heavy servers

Prefect

Le MLOps, ce n'est pas seulement emballer du code dans Docker. C'est s'assurer que les données coulent de manière fiable et que les modèles restent performants dans le temps.

Prefect transforme les scripts Python en workflows robustes, capables de :

- s'auto-réparer (Retries)
- se déclencher seuls (Scheduling)
- surveiller la santé de vos prédictions (Drifting).

Décorateur Prefect

`@flow` (L'Orchestrateur) : le point d'entrée du pipeline.

- `name` : Pour donner un nom clair dans l'interface.
- `log_prints=True` : Capture tous les `print()` pour les afficher dans les `logs Prefect`.
- `description` : Pour documenter ce que fait le flux.

Peut lancer plusieurs tâche

Décorateur Prefect

`@task` (L'Unité de Travail) : une étape spécifique du flow

- `retries` : Nombre de tentatives en cas d'erreur.
- `retry_delay_seconds` : Temps d'attente entre deux essais.
- `tags` : Pour filtrer ou limiter la concurrence (ex: ["toto"]). Si toto =2 alors `prefect` pourra lancer jusqu'à deux jobs en parallèle.
- `cache_key_fn` : Pour ne pas recalculer une étape si les données n'ont pas changé.

Une opération unitaire qui peut nécessiter les options des tâches.

exemple

```python
@flow(name="Demo Concurrence", log_prints=True)
def pipeline_calcul():
    print("Envoi de 10 tâches à l'API ")

    futures = []
    # 1. On lance les tâches (file d'attente)
    for _ in range(10):
        fut = appeler_api.submit(a, b)
        futures.append(fut)

    # 2. attend que chaque tâche soit terminée
    for f in futures:
        f.result()

    print("Toutes les tâches sont terminées !")
```




```python
@task(tags=["calcul_max"], retries=3, retry_delay_seconds=10)
def appeler_api(a, b):
    debut = datetime.now().strftime("%H:%M:%S")

    with httpx.Client(timeout=10.0) as client:

        # On appelle l'API
        r = client.get(f"http://localhost:8000/calcul?a={a}&b={b}")
        res = r.json()["resultat"]

        # On affiche l'heure de FIN pour bien voir les groupes
        fin = datetime.now().strftime("%H:%M:%S")

        print(f"Début {debut} -> Fin {fin} | Résultat: {res}")

        return res
```

https://github.com/DeVerMyst/docker-compose-prefect

Développement local

`process.serve()` : lance un déploiement en local, c'est à dire dans le terminal. Si on éteint le terminal alors l'automation s'arrête.

Il faut qu'un serveur `prefect` soit actif : `server start`

`Identité :` name, description, tags

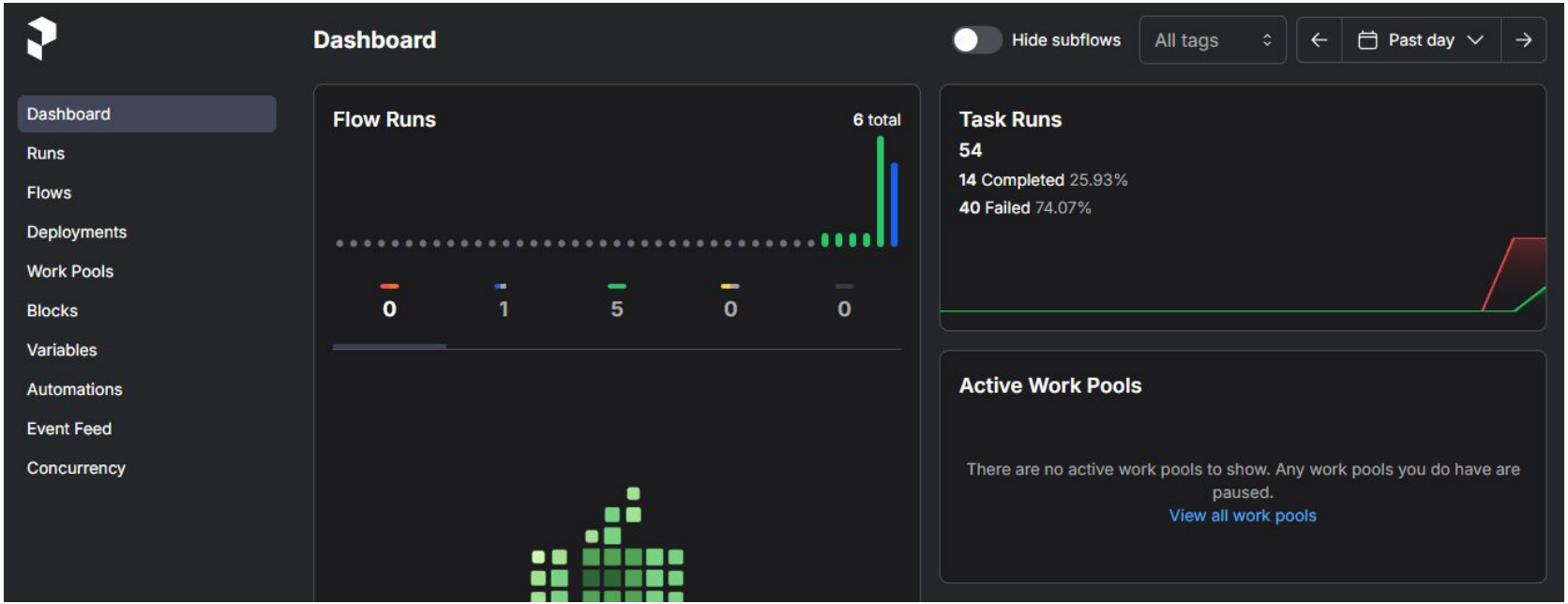
`Cadencement :`

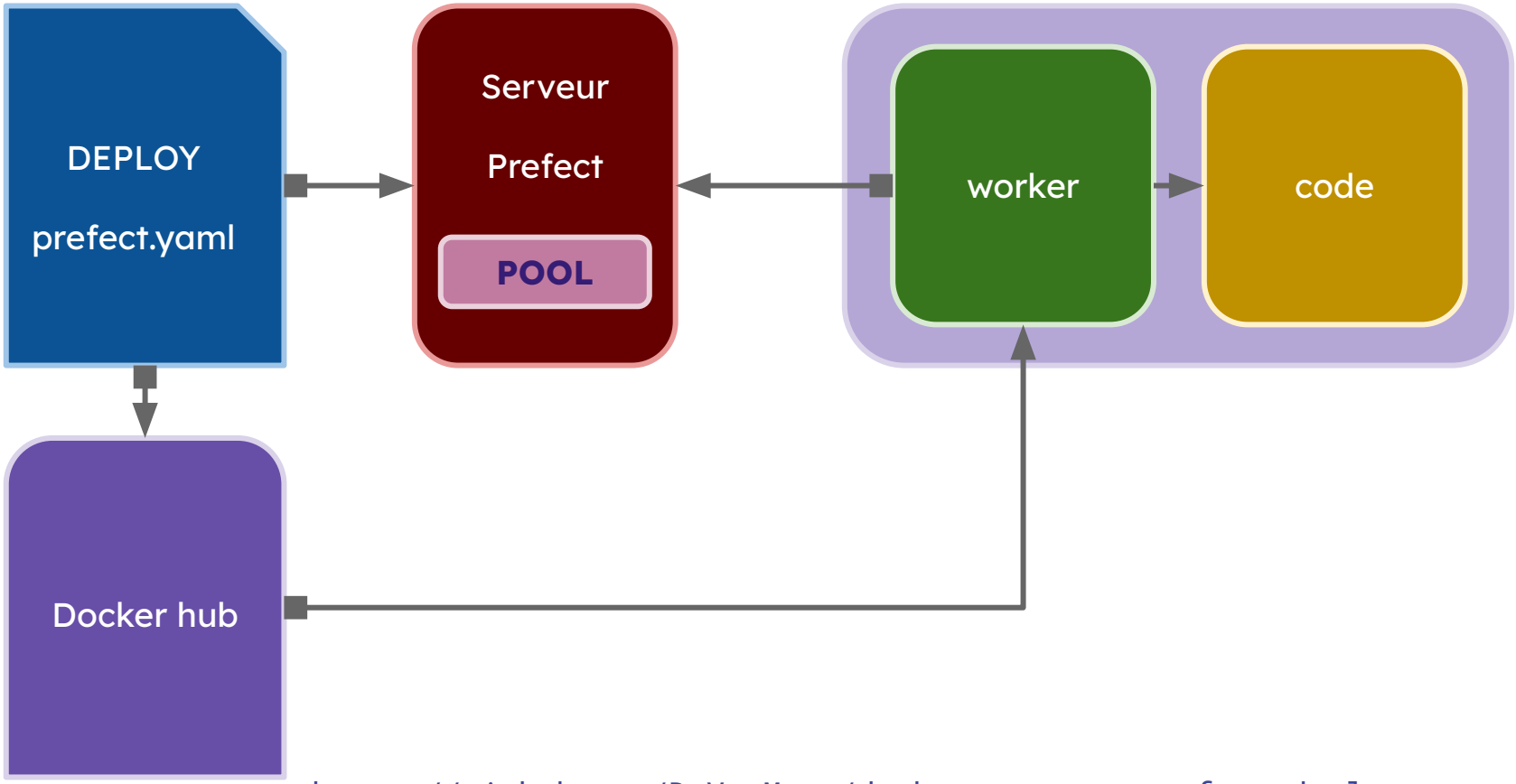
- interval (en seconde)
- cron (cron="0 12 * * * → tous les jours à midi)
- rrule (répétition plus complexes)

On peut aussi passer des paramètres et imposer des limites sur le nombre de runs

```
if __name__ == "__main__":
    # "Deployment" avec un interval de 10s
    pipeline_calcul.serve(
        name="mon-deployment-automatique",
        interval=20 # le scheduling (en secondes)
    )
```

UI Prefect :4200/dashboard





https://github.com/DeVerMyst/docker-compose-prefect-deploy

Le déploiement

Pour un vrai déploiement on enlève les “serve” du code et on utilise la commande : `prefect deploy` dans le terminal.

Cette commande :

- build les images (pré build par précaution)
- push sur docker hub
- donne les règles pour le pull de l’image
- configure le worker

yaml et non yml

```
# 1. On build les images manuellement ou avec le CI/CD
docker build -t account/api-flow:latest .
docker push account/api-flow:latest

# 2. On lance le serveur
docker-compose up

# 3. déploiement du flow (création du worker, connection au serveur et déploiement)
prefect work-pool create "docker-pool" --type docker
prefect config set PREFECT_API_URL="http://localhost:4200/api"
prefect deploy --all

# 4. On démarre le pool
prefect worker start --pool 'docker-pool'
```

```
prefect init
```

Avec les bonnes options initialise le prefect.yaml

Prefect.yaml

```yaml
name: docker-compose-prefect-deploy
prefect-version: 3.6.21

build:
- prefect_docker.deployments.steps.build_docker_image:
    id: build_image
    requires: prefect-docker>=0.3.1
    image_name: devermyst/api-flow
    tag: latest
    dockerfile: Dockerfile
```

On définit le nom du projet et donne les règles pour le build :

- quel Dockerfile
- quel nom pour l'image
- quel tag pour l'image

Prefect.yaml

```yaml
push:
- prefect_docker.deployments.steps.push_docker_image:
    requires: prefect-docker>=0.3.1
    image_name: '{{ build_image.image_name }}'
    tag: '{{ build_image.tag }}'

pull:
- prefect.deployments.steps.set_working_directory:
    directory: /app
```

L'image sur docker hub aura le même nom et le même tag que l'image buildé

Comme dans notre Dockerfile on met tout dans le dossier `/app` on donne comme règle de pull que le worker sera dans le dossier `/app`

Prefect.yaml

```yaml
deployments:
  - name: "calcul-final-v5"
    entrypoint: "flows/task.py:mon_flow_calcul"
    work_pool:
      name: "docker-pool"
      job_variables:
        image: devermyst/api-flow:latest
        env:
          PREFECT_API_URL: http://host.docker.internal:4200/api
    parameters:
      a: 10
      b: 20
    schedule:
      cron: "0 9 * * *"
```

On peut avoir plusieurs déploiement depuis un seul `prefect.yaml`
ici, la fonction mon_flow_calcul du fichier task.py du dossier flows

work_pool : pour séparer (par exemple si on a plusieurs serveur)

Il faut lui donner le nom de l'image et surtout lui donner la possibilité de communiquer en dehors du conteneur (hos)

parameters : des variables à transmettre

schedule : le schedule

Dockerfile du worker (`flows/task.py`)

```
FROM prefecthq/prefect:3-python3.11

# On force la création du dossier
RUN mkdir -p /app
WORKDIR /app

# On installe les dépendances
COPY pyproject.toml .
RUN pip install --no-cache-dir .

# On copie tout le contenu du dossier actuel dans /app
COPY . /app

# On s'assure que Python regarde dans /app
ENV PYTHONPATH=/app
```

Contrairement à d'habitude on ne crée pas une image docker pour exécuter du code.

Prefect s'en chargera, on a juste besoin d'avoir le code dans une image

Docker-compose pour le serveur Prefect

```yaml
services:
  ### 1. LA BASE DE DONNÉES
  database:
    image: postgres:15-alpine
    restart: always
    environment:
      - POSTGRES_USER=prefect
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=prefect
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - prefect-network
```

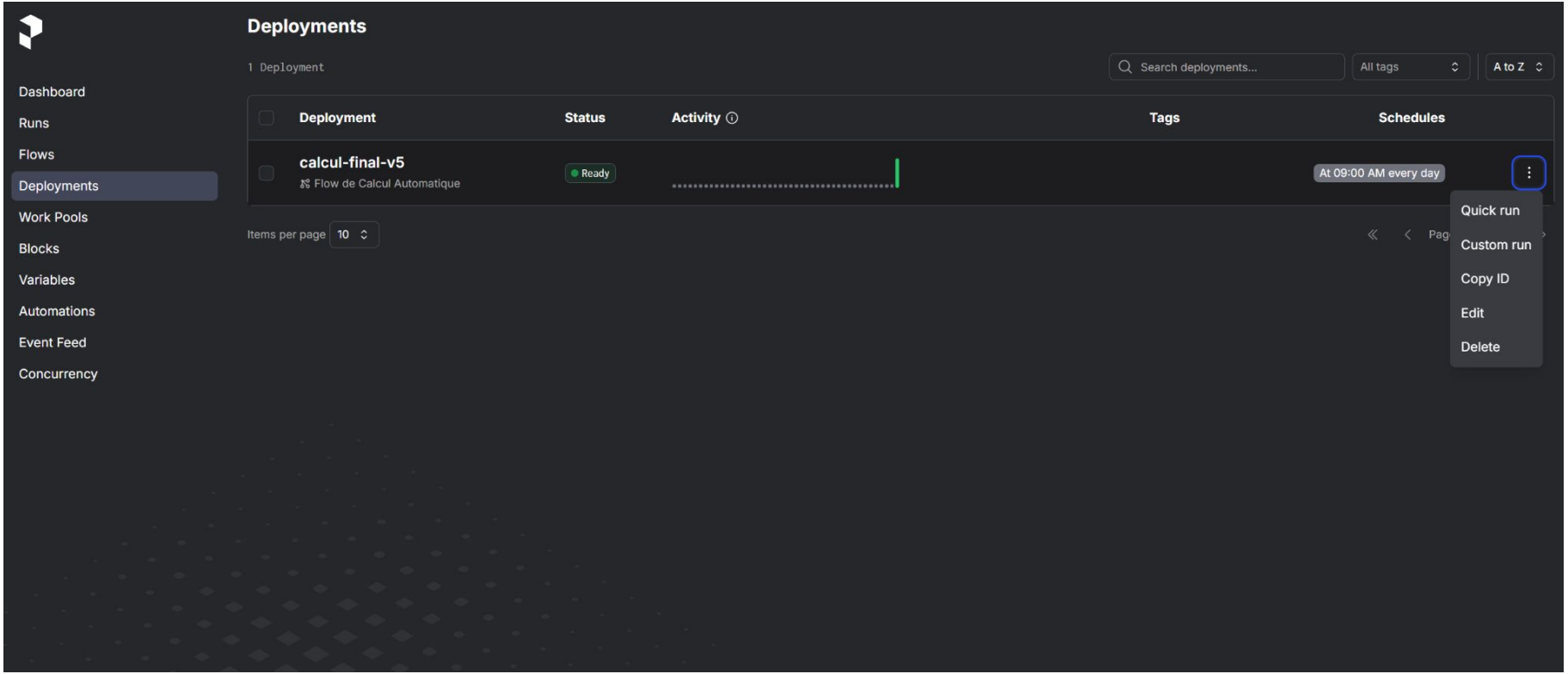
Le docker-compose aura 2+1 parties pour un seul worker:

- la base de données de prefect pour suivre les déploiements
- la configuration du serveur prefect
- la configuration du worker

```yaml
  ### 2. LE SERVEUR PREFECT
  server:
    image: prefecthq/prefect:3-latest
    restart: always
    volumes:
      - prefect_data:/root/.prefect
    entrypoint: ["prefect", "server", "start"]
    environment:
      # Connection DB
      - PREFECT_API_DATABASE_CONNECTION_URL=postgresql+asyncpg://prefect:${POSTGRES_PASSWORD}@database:5432/prefect#
      # Autorise le serveur à écouter toutes les interfaces réseau du conteneur (extérieur au conteneur)
      - PREFECT_SERVER_API_HOST=0.0.0.0
      # 127.0.0.1 pointait vers le conteneur lui-même, pas le serveur Prefect
      - PREFECT_API_URL=http://server:4200/api
      # L'URL que l'UI utilisera
      - PREFECT_UI_API_URL=http://localhost:4200/api
    ports:
      - "4200:4200"
    depends_on:
      - database
    networks:
      - prefect-network

networks:
  prefect-network:
    driver: bridge

volumes:
  db_data:
  prefect_data:
```


Deployments
1 Deployment
Search deployments...
All tags
A to Z
Dashboard
Runs
Flows
Deployments
Work Pools
Blocks
Variables
Automations
Event Feed
Concurrency
Deployment
Status
Activity
Tags
Schedules
calcul-final-v5
Flow de Calcul Automatique
Ready
At 09:00 AM every day
Quick run
Custom run
Copy ID
Edit
Delete
Items per page
10